Object-Oriented

Programming Assignment 1

Course Instructor : Ma'am Hina Iqbal

: hina.iqbal@lhr.nu.edu.pk

Teacher Assistant (TA) : Taimoor Shakeel: l226822@lhr.nu.edu.pk

Section : BCS-2F

Note :

- **The last question is not mandatory but carries bonus marks, which will be decided later**

- **No marks will be awarded for solutions that use static arrays. All data structures must use dynamic memory allocation.**
- **Proper memory management is mandatory. Programs must not contain dangling pointers, memory leaks, or unsafe memory access.**
- **The assignment will be evaluated carefully for plagiarism and AI-generated solutions. Students must complete the work independently.**
- **The deadline is strict. No extensions will be granted under any circumstances. Students are advised to start early.**

Question 1: Heisenberg's Underground Menu Management System

You have been recruited as a Backend Developer by *BlueSky Deliveries*, an underground food distribution platform operating quietly in the Breaking Bad universe. Much like Heisenberg's operation, the data you receive is messy, unorganized, and dangerous if mishandled. Your job is to clean it up with precision.

The system must read menu data from a text file, process it into appropriate data structures, and provide sorting and searching functionality

Your Tasks

1. Read the menu data from a text file and initially store it in a 2D array used strictly for file input handling.

2. From this file-based 2D array, dynamically create a new 2D string array.

   The dynamically created 2D array must follow these rules:

   - Memory must be allocated dynamically (manual control , no shortcuts).
   - Each row represents one menu item.
   - The last column of each row must contain a null character to mark the end of that row.

3. Organize the menu items using the following priority order (precision matters):

- Cuisine Type (Mexican, American, Fusion, Experimental, etc.)
- Restaurant Name (A–Z)
- Category (Appetizer, Main Course, Dessert, etc.)
- Price (Low to High)

4. Implement a search feature that allows users to enter a cuisine type and view all related menu items.
    - For example, entering Mexican should display Mexican*,* Mexican-Street*,* Mexican-Fusion items together in a single sorted list.
    - Entering Experimental should display *Experimental,* Experimental-Fusion*,* Experimental-Blue items together.
5. Display the correctly sorted and filtered results on the console.

    If your output is wrong, the whole operation fails.

6. Export the organized menu data to an output file in a readable format. *(Bonus marks consider it extra profit.)*

Data Format (Unorganized)

The input file contains comma separated values in the following format:

Category, Cuisine Type, Item Name, Price, Restaurant Name, Cook Name,

Calories **Example Data (Random Order)**

Main Course, Experimental-Blue, Crystal Burger, 1500, Heisenberg Kitchen, Walter White, 1800

Dessert, Experimental, Blue Ice Pops, 350, Pinkman Treats, Jesse Pinkman, 500

Appetizer, Mexican-Street, Sloppy Nachos, 400, Saul’s Shady Snacks, , 700 Main

Course, American-BBQ, Smoked Ribs, 1200, Schrader Grill, Hank Schrader, 1400

Dessert, Mexican, Churros, 250, Los Pollos Hermanos, , 500

Important Notes

- Some menu items may not include Cook Name or Calories. Your program must handle missing data cleanly and safely.
- The dynamic 2D string array must be created from the 2D array used for reading file input.
- The last column of the dynamically created 2D array must contain a null character — no exceptions.
- Displaying correct results on the console is mandatory.
- Storing the final organized result back into a file earns bonus marks.

Question 2 : Shopping Cart Management System

You are required to design and manage a Shopping Cart Management System by implementing the following functions. Each function has a specific role in managing cart items and their attributes.

addItem
Adds a new item to the shopping cart.

removeItem
Removes an existing item from the cart.

addAttribute
Adds a new attribute or property to items (for example, color, size, or warranty). The attribute data must be passed through the function parameters.

removeAttribute
Removes a specified attribute from items in the cart. This function must support item ranges (such as 1–4). If the cart contains four items and only item 1 is specified, the attribute should be removed only from item 1.

getItemInfo
Returns the complete information of a specific item. The item's position in the cart can be used to retrieve details for operations such as addAttribute and removeAttribute.

clearCart
Removes all items from the cart and leaves it empty.

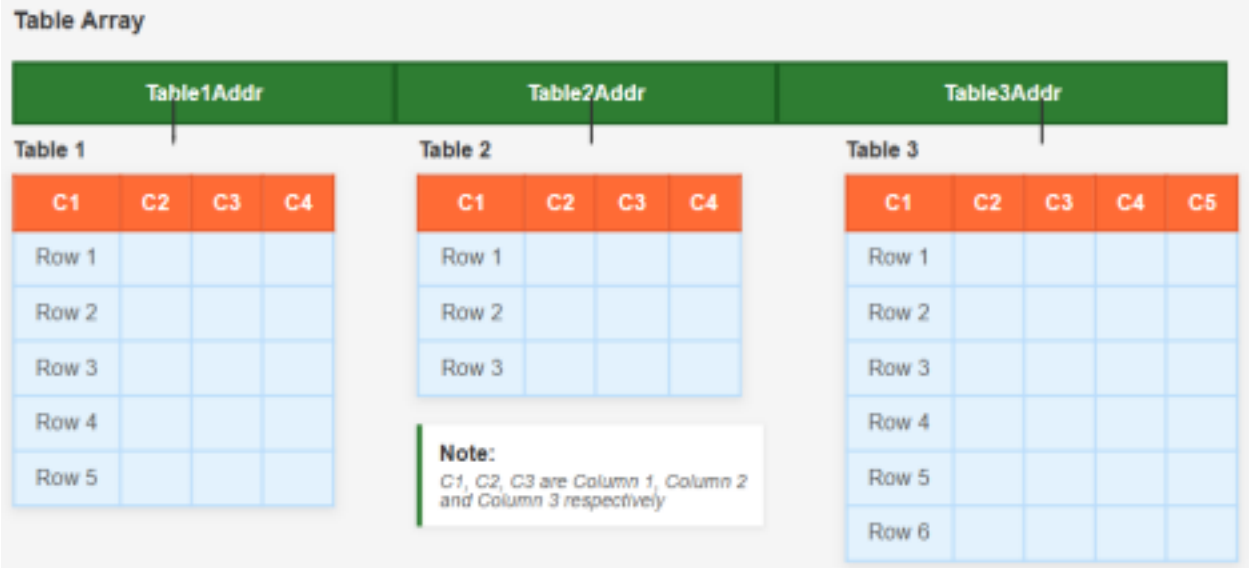
sortCartByAttr
Sorts the cart based on a specified attribute. For text values, sorting should be alphabetical (A–Z). For numeric values, sorting should be in ascending order (0–9). For prices, sorting should be from lowest to highest.

totalCartValue
Calculates and returns the total price of all items in the cart using the given price attribute name.

avgCartValue
Calculates and returns the average price of all items in the cart using the given price attribute name.

filterByAttribute
Retrieves items from the cart based on attribute values. If no specific value is provided, all items are returned. If a specific value is provided (for example, Electronics), only

matching items are returned. If a price range is provided (for example, 100–500), only items within that range are returned.



Question 3 : The Inventory Cleanup (E-commerce)

The Scenario: You are managing a warehouse database for a small electronics store. The store uses a fixed-size 2D grid to track stock levels for different categories of items across various shelves. Many slots in the grid are empty (represented by **0**). To save memory and generate a "Pick List" for a warehouse robot, you need to condense this data.

The Task: Write a program that takes the warehouse grid (the input matrix) and produces a "compact list" (the output array). This new structure should only contain the actual stock counts for each category, ignoring all empty slots. Each row in your new structure will represent a shelf, but will only be as long as the number of items actually present on that shelf.

Input Matrix: 2D Array

2	3	1	0	0	0
0	0	0	1	1	0
1	0	0	0	0	0
1	1	1	2	0	2
5	0	0	0	10	0

Output Array:

2	3	1		
1	1			
1				
1	1	1	2	2
5	10			

Bonus Question : Thesaurus-Based Writing Assistant

Paraphrasing is commonly used in writing tools to improve clarity and avoid repetition. You are developing a simple Thesaurus-Based Writing Assistant that helps users replace words with their synonyms.

You are required to write a function that takes a character array as input. After processing the data, the function should dynamically create a 2D string array and return a double pointer. The function prototype is given below:

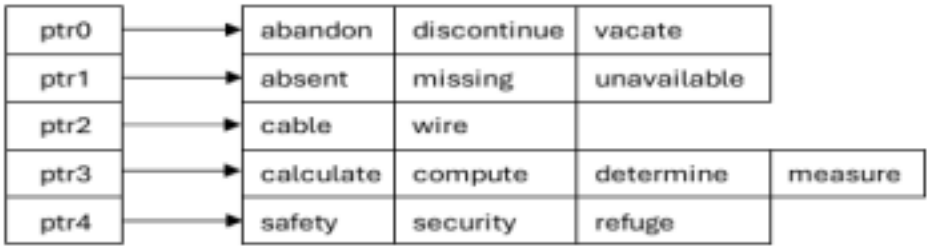
string CreateDynamicArray(char* ws);**

The character array will contain words and their synonyms. Words belonging to the same group are separated by a hash symbol (#).

Example input string:
"abandon discontinue vacate#absent missing unavailable#cable wire#calculate compute determine measure#safety security refuge"

In this character array, each group represents a word followed by its synonyms.

a) Your first task is to dynamically create a 2D string array where each row stores one word group and its synonyms in separate column like the picture below.



b) Your second task is to take a word as input from the user and display its last synonym. The user will always enter the first word of a group.
For example, if the user enters absent, the program should display unavailable.

c) Write a proper main function to demonstrate and test the functionality of your program.

Sample Input:
Enter word to paraphrase: absent

Sample Output:
Replaced word: unavailable


BASANT HOLIDAYS
OOP
ASSIGNMENT